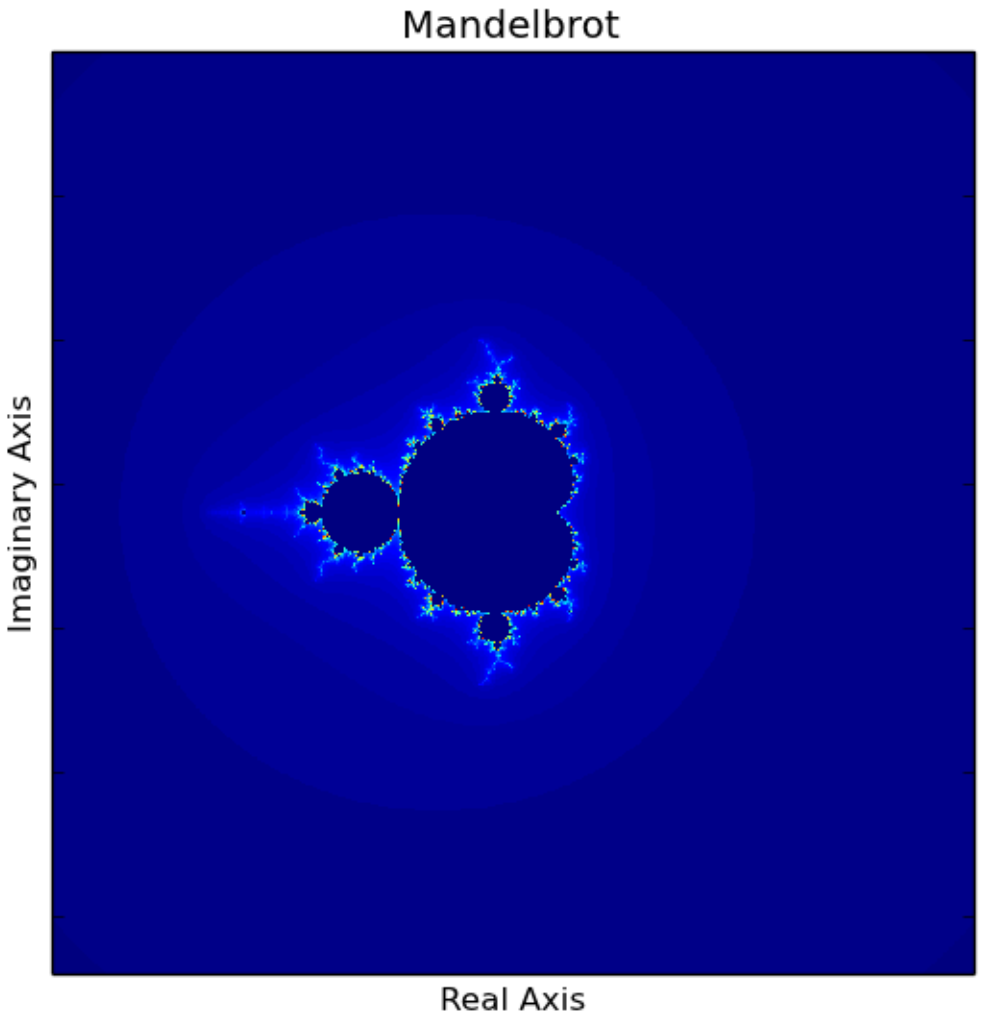
COMP 1012 – Scientific Programming



Labs

- All labs are 90 minutes long.
- You must attend the lab in which you registered.
- There is a total of 11 labs worth 1 mark each.
- The maximum number of lab marks is 10.
- Can miss 1 lab and still get full marks for the labs.
- You should use your full name while you are in zoom session.
- You should attend the labs via your ICM email.

- For this course we will use the Anaconda Python 3.6 64-bit distribution.
- This software is free.
- Follow the installation instructions posted on Moodle (somewhat out of date).
- There are versions for Mac OSX and Windows, download and install the correct version (3.6, 64-bit) if you wish to use your own computer to complete the assignments and labs.
- As Python is installed on the computers in the open area labs you do not need to own a computer to complete this course.

Your first python program

- The first program most people write displays the words `Hello World!` on the screen.
- Click on the icon for Anaconda Navigator to start the IDE, then click on the *Launch* button for spyder-app.
- The lower pane of the window acts as an interactive shell that executes program statements as they are typed.
- A prompt of the form **`In [1]:`** is displayed to tell you when Python is ready for you to type in a statement.
- On the same line as the prompt enter the statement to be executed.

Your first python program

- `In [1]: print('Hello World!')`
  `Hello World!`
- **`print('Hello World!')`** is a statement.
- This statement is executed after it is entered and the value `Hello World!` is displayed on the screen.
- The value `'Hello World!'` is a string literal.
- All string literals are characters enclosed in quotes.
- A character is anything you can type on your computer's keyboard.

- Python has several types of literals
- 'hello' and "hello" are both string literals where the enclosing quotes must be the same
- 'hello" and "hello' are not valid string literals as the enclosing quotes do not match
- "I'm hungry!" is a string literal that contains the characters **I'm hungry!**
- In order for ' (single quote) to appear in a string literal the literal must be enclosed by " (double quote) as shown above
- In order for " (double quote) to appear in a string literal the literal must be enclosed by ' (single quote) as shown below

```
'He said "Here I am."'
```

- Another way to do this is to use the escape character \
- ```
  In [2]: print('I\'m hungry!')
  I'm hungry!
  ```
- ```
  In [3]: print("He said \"Here I am.\"")
  He said "Here I am."
  ```
- Other commonly used escape sequences are \t for tab and \n for newline.
- ```
  In [4]: print('x\tx^2\tx^3')
  x	x^2	x^3
  ```
- ```
  In [5]: print('hello\nworld')
  hello
  world
  ```

- 1, -17, 214158 are examples of integer literals
- An integer is a number that does not have a fractional part.
- 3.14159, -2., 1.234E12 are examples of real numbers or floating point literals and must contain a decimal point
- A real or floating point number has a fractional part.
- 1.234E12 is the number 1.234×10^{12}
- 2.37e-5 is the number 2.37×10^{-5}

- True and False are known as boolean literals (more on this later)

```
In [1]: print(123, -7.23, 0)
123 -7.23 0
In [2]: print('numbers: ', 123, 7.5)
numbers: 123 7.5
In [3]: print('numbers: ' +
        repr(123) + ' ' + repr(7.5))
numbers: 123 7.5
```

- repr(123) gives the string equivalent '123'
- The + operator joins together (catenates) two strings into one longer string.
- The 3 + operators shown above catenate 4 strings.

Operators

- There are a number of arithmetic operators:

Operator	Name	Example
**	Exponentiation	2**4
- +	Unary minus and plus	-16, +2
* / // %	Multiplication, division, remainder	2*5, 12/9, 12//9, 12%9
+ -	Addition, subtraction	22 + 1001, 97 - 53

- 2**4 means 2^4 which equals 16
- / and // both perform division

- There are 2 division operators: / and //
- / gives a result with a fractional part

```
In [2]: 12/4
Out[2]: 3.0
In [3]: 15/6
Out[3]: 2.5
In [4]: 1./7.
Out[4]: 0.14285714285714285
```

- For integers // gives an integer result with no fractional part

```
In [5]: 12//4
Out[5]: 3
In [6]: 15//6
Out[6]: 2
In [7]: 1//6
Out[7]: 0
```

- For floating point numbers // gives a floating point result where the fractional part is 0

```
In [1]: 1.//7.
Out[1]: 0.0
In [2]: 12.//7.
Out[2]: 1.0
```

- Integers and floating point numbers may be mixed together in an expression. When this is done the integer is converted to a float.

```
In [3]: 12./7
Out[3]: 1.7142857142857142
In [4]: 12/7.
Out[4]: 1.7142857142857142
In [5]: 12.//7
Out[5]: 1.0
In [6]: 12//7.
Out[6]: 1.0
```

Types

- Python supports the following types:
  - str (a sequence of characters enclosed in quotes)
  - int (an integer containing any number of digits)
  - float (a real or floating point number)
  - bool (True or False)
- There are other types in python that will be discussed later.

Variables and Assignment Statements

- A variable holds a value of some type.
- The type of a variable is the type of the value it holds.
- `In [1]: PI = 3.14159`
- PI is a variable whose value is 3.14159 and whose type is float
- `In [2]: count = 1000`
- count is a variable whose value is 1000 and whose type is int

- The type of a variable may change as new values are assigned to it.

```
In [1]: total = 0
In [2]: print(type(total))
<class 'int'>
```

- total is an **int**

```
In [3]: total = 1299.99
In [4]: print(type(total))
<class 'float'>
```

- total is now a **float**

Expressions

- An expression is a combination of one or more variables or literals and operators.

```
In [1]: PI = 3.14159
In [2]: radius = 2
In [3]: print(PI * radius**2)
12.56636
In [4]: area = PI * radius**2
In [5]: print(area)
12.56636
```

```
In [1]: a = 5
In [2]: b = 7
In [3]: c = 10
In [4]: d = 15
In [5]: print(a + b * c)
75
In [6]: print((a + b) * c)
120
In [7]: print(b // a * c)
10
In [8]: print(d % b)
1
In [9]: print(a * -d)
-75
In [10]: print(5 * 2**a)
160
In [11]: print(2**2**3)     # 2^8 = 256, not 4^3 = 64
256
```

Operator Precedence

Operator	Precedence
**	highest
- + (unary operators)	
* / // %	
+ -	
=	lowest

- Operators of the same precedence are evaluated from left to right, except for ** which is evaluated from right to left.
- Parentheses override operator precedence.

- If I have $4.00 in cash and a bag of chips costs 75 cents ($0.75), how many bags of chips can I buy and how much money will I have left over?

```
In [1]: cash = 4.0
In [2]: chips = 0.75
In [3]: bags = cash // chips
In [4]: print(bags)
5.0
In [5]: leftOver = cash % chips
In [6]: print(leftOver)
0.25
```

Naming Conventions

- The names of all variables, scripts, functions, modules, etc. should be meaningful.
- Each name should be short and concise.
- Variable names usually start with a lowercase letter, and may contain uppercase letters, underscores and digits. Spaces are NOT allowed.
- Mixed case is used when a name is made up of more than one word and is known as camel case.
- total, average, interestRate, xValues are examples of good variable names.

CONSTANTS

- A constant is a variable whose value never changes. There is always 60 minutes per hour, 1000 meters per kilometer, etc.
- To indicate that a variable is a constant use only uppercase letters and underscores in its name.

```
In [1]: MINUTES_PER_HOUR = 60
In [2]: METERS_PER_KILOMETER = 1000
```

- Always use named constants rather than literals for values that (should) never change, especially if they are used more than once in a script / program.

Scripts (Programs)

- So far we have have typed all of our statements into the shell after the `In [1]:` prompt and the statements were executed immediately.
- Each time we want to execute the same statements we must type them in.
- It would be much better if we could store our statements into a file (script) and then execute the script each time we want the statements executed.

Celsius to Fahrenheit

- fahrenheit = (9/5) × celsius + 32
- In anaconda select File -> New File
- Type in the following:

```
# Given a temperature in degrees
# celsius convert it to degrees
# fahrenheit
celsius = 20
fahrenheit = (9/5)*celsius + 32
print("celsius = " + repr(celsius) +
", fahrenheit = " + repr(fahrenheit))
```

- Save the file as toFahrenheit.py

Celsius to Fahrenheit

- To execute the script (run the program) press F5.
- The output should be as follows:

```
celsius = 20, fahrenheit = 68.0
```

- Each line beginning with # is a comment.
- Everything from the # to the end of the line is not executed.
- Comments are used to explain what the program does and the use of each variable.
- A comment may be placed at the end of an executable statement.

Formatted Output

```
# Given a temperature in degrees
# celsius convert it to degrees
# fahrenheit
celsius = 20
fahrenheit = (9/5)*celsius + 32
print("celsius = %g, fahrenheit = %g" %
  (celsius, fahrenheit))
```

- The output is:

```
celsius = 20, fahrenheit = 68
```

Formatted Output

- When using **print** to display output we can catenate a number of strings or we can use format codes.
- A format code is of the form:

  % flags width code
- The optional flags are:

  '0' pad with leading zeros

  '-' left justification (overrides '0')

  ' ' (space) a space should appear before a positive number

  '+' a sign (+ or -) should appear before a number (overrides ' ')

- To output an integer the optional width is an integer specifying the number of character positions in which to display the integer.
- To output a real number the width is a real number. The integer part is the number of character positions in which to display the real number. The fractional part is the number of decimal places in the output.

✓7.2 => 7 positions with 2 digits after the decimal point (must include space for the decimal point)

.4 => use as many positions as necessary but have 4 digits after the decimal point.

- The codes are:

  d - signed decimal integer

  i - signed decimal integer

  e - floating point with exponent

  E - floating point with exponent

  f - floating point without exponent

  F - floating point without exponent

  g - floating point general, use e or f as appropriate

  G - floating point general, use E or F as appropriate

  c - character format

  s - string format using str()

  r - string format using repr()

- %[width][.precision]type
- Examples:

```
"%4d" % -27 ➔ ' -27'
"%-3d" % 42 ➔ '42 '
"%8.2e" % 17.5 ➔ '1.75e+01'
"%-10.3E" % 0.123 ➔ '1.230E-01 '
"%-8.2f" % 17.5 ➔ '17.50   '
"%.4F" % 0.123 ➔ '0.1230'
"%g" % 1.234 ➔ '1.234'
"%G" % 12.34e15 ➔ '1.234E+16'
"%4c" % 65 ➔ '   A'
"%c" % "f" ➔ 'f'
"%s" % "hello" ➔ 'hello'
```

Input From The Keyboard

- To get input from the keyboard use the function *input*
- *input* returns what is typed on the keyboard as a **string**
- e.g.

```
data = input('Enter text: ')
Enter text: hello
In [1]: data
'hello'
```

- The value assigned to *data* is a string!

- Suppose you need an integer or a real number to use in a program.
- The string returned by *input* must be converted to the correct type.

```
In [1]: prompt = "Enter an integer: ”
In [2]: data = int(input(prompt))
Enter an integer: 123
In [3]: print(data)
123
In [4]: print(type(data))
<class 'int'>
```

- The value assigned to *data* is an integer number.

```
In [1]: prompt = "Enter a real number: ”
In [2]: data = float(input(prompt))
Enter a real number: 12.34
In [3]: print(data)
12.34
In [4]: print(type(data))
<class 'float'>
```

- There is a difference between the string '123' and the integer 123.
- '123' is a string of characters, it is NOT a number
- 123 is an integer number.

- Consider throwing a ball straight up into the air. At what point in time will it return to the position from which it was thrown?
- This depends on how hard it was thrown (it's initial velocity) and the force of gravity.
- The equation we need is

  $y(t) = v_0 t - \frac{1}{2} g t^2$

  where v_0 is the initial velocity in m/s, g is the force of gravity (9.81 m/s^2) and y(t) is the height of the ball in meters at time t in seconds.
- When y(t) = 0 the ball is at the position from which it was thrown. This happens twice, once when the ball is thrown up at time t = 0, and once when the ball returns to the position from which it was thrown.
- We need to find the value of t when the balls returns to the position from which is was thrown.

- Solve for $y(t) = 0$
- $v_0t - \frac{1}{2}gt^2 = 0$
- $v_0t = \frac{1}{2}gt^2$
- $v_0 = \frac{1}{2}gt$
- $2v_0/g = t$
- Write a program to find the value of t (in seconds) given the initial velocity (in m/s).
- As we are solving an equation **t**, **g**, **v0** are meaningful variable names

```
# in class demo of verticalMotion.py

G = 9.81
v0 = float(input(
     "Enter the initial velocity in m/s: "))
t = 2 * v0 / G
print('For an initial velocity of %.1f m/s'
      ' the ball returns to its '
      'initial position after %g seconds.'
      % (v0, t))
```

Math Functions

- To use math functions such as sin, square root, exp, log, etc. the math module must be imported

```
In [1]: import math
```

Refer to the functions using math as the prefix, such as math.sin(x) and math.log(y)

```
In [2]: import math as mm
```

Refer to functions using mm as the prefix, such as mm.sin(x) and mm.log(y)

```
In [3]: from math import *
```

Refer to functions without a prefix, such as sin(x) and log(y)

```
In [4]: from math import sin
```

Import only the sin function from math, refer to it as sin(x)

Some of the math functions are:

- sqrt(x) return the square root of x
- pow(x, y) return x^y
- exp(x) return e^x
- log(x) return $\log_e x$
- log(x, b) return $\log_b x$
- log10(x) return $\log_{10} x$
- sin(x), cos(x), tan(x) return sin(x), cos(x), tan(x) where x is specified in radians (π radians = 180 degrees)
- asin(x), acos(x), atan(x) return a value measured in radians

Examples:

```
In [1]: import math
>>> print(math.sqrt(9 + 16))
5.0
In [2]: print(math.exp(3.5))
33.11545195869231
In [3]: print(math.sin(math.pi/2))  # sine of 90°
1.0
In [4]: print(math.pow(2.5, 3)) # same as 2.5**3
15.625
In [5]: print(math.log(10000,10))
4.0
In [6]: print(math.log10(10000)) # same as above
4.0
In [7]: print(math.pi)
3.141592653589793
```

- Notice that math.pi gives the value of π

Other Useful Functions

```
In [1]: print(abs(-17.5))
17.5
In [2]: print(min(5, -17, 12))
-17
In [3]: print(max(25.2, 1e10, -99, 327.34))
10000000000.0
In [4]: print(type(123))
<class 'int'>
In [5]: print(type(123.))
<class 'float'>
In [6]: print(type('123'))
<class 'str'>
```